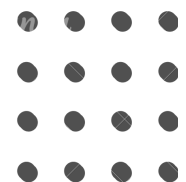




VMES Grinds





Leetcode: 3652
Best Time to Buy and Sell Stock
using Strategy





3652. Best Time to Buy and Sell Stock using Strategy

Solved

Medium

Topics

Companies

Hint

You are given two integer arrays `prices` and `strategy`, where:

- `prices[i]` is the price of a given stock on the i^{th} day.
- `strategy[i]` represents a trading action on the i^{th} day, where:
 - `-1` indicates buying one unit of the stock.
 - `0` indicates holding the stock.
 - `1` indicates selling one unit of the stock.

You are also given an **even** integer `k`, and may perform **at most one** modification to `strategy`. A modification consists of:

- Selecting exactly `k` **consecutive** elements in `strategy`.
- Set the **first** `k / 2` elements to `0` (hold).
- Set the **last** `k / 2` elements to `1` (sell).

The **profit** is defined as the **sum** of `strategy[i] * prices[i]` across all days.

Return the **maximum** possible profit you can achieve.

Note: There are no constraints on budget or stock ownership, so all buy and sell operations are feasible regardless of past actions.



Example 1:

Input: prices = [4,2,8], strategy = [-1,0,1], k = 2

Output: 10

Explanation:

Modification	Strategy	Profit Calculation	Profit
Original	[-1, 0, 1]	$(-1 \times 4) + (0 \times 2) + (1 \times 8) = -4 + 0 + 8$	4
Modify [0, 1]	[0, 1, 1]	$(0 \times 4) + (1 \times 2) + (1 \times 8) = 0 + 2 + 8$	10
Modify [1, 2]	[-1, 0, 1]	$(-1 \times 4) + (0 \times 2) + (1 \times 8) = -4 + 0 + 8$	4

Thus, the maximum possible profit is 10, which is achieved by modifying the subarray [0, 1].

Constraints:

- $2 \leq \text{prices.length} == \text{strategy.length} \leq 10^5$
- $1 \leq \text{prices}[i] \leq 10^5$
- $-1 \leq \text{strategy}[i] \leq 1$
- $2 \leq k \leq \text{prices.length}$
- k is even

Example 2:

Input: prices = [5,4,3], strategy = [1,1,0], k = 2

Output: 9

Explanation:

Modification	Strategy	Profit Calculation	Profit
Original	[1, 1, 0]	$(1 \times 5) + (1 \times 4) + (0 \times 3) = 5 + 4 + 0$	9
Modify [0, 1]	[0, 1, 0]	$(0 \times 5) + (1 \times 4) + (0 \times 3) = 0 + 4 + 0$	4
Modify [1, 2]	[1, 0, 1]	$(1 \times 5) + (0 \times 4) + (1 \times 3) = 5 + 0 + 3$	8

Thus, the maximum possible profit is 9, which is achieved without any modification.



Explanation





Sliding Window Strategy Optimizer

Visualizing maxProfit delta calculation

☐ Target 0 Zone ☐ Target 1 Zone ☐ Leaving ☐ Entering ☐ Middle (Shifting)

Prices	10	20	5	30	15	25	10	30
Strategy	1	0	1	0	0	1	1	0
Window								
Target								
	0	1	2	3	4	5	6	7



Play



Reset



Step 7 of 7



Initial Approach





- 1. Calculate the original profit without any change!**
- 2. Modify the strategy with a loop**
- 3. Recalculate the profit again and update if relevant**
- 4. Done**



1. Calculate the original profit without any change!
2. Modify the strategy with a loop
3. Recalculate the profit again and update if relevant
4. Done

```
class Solution:
    def maxProfit(self, prices: List[int], strategy: List[int], k: int) → int:
        n = len(prices)

        def calculate_profit(strat):
            total = 0
            for i in range(len(strat)):
                total += strat[i] * prices[i]
            return total

        original_profit = calculate_profit(strategy)
        max_profit = original_profit

        for start in range(n - k + 1):
            modified_strategy = strategy.copy()

            half_k = k // 2

            for i in range(start, start + half_k):
                modified_strategy[i] = 0

            for i in range(start + half_k, start + k):
                modified_strategy[i] = 1

            new_profit = calculate_profit(modified_strategy)

            max_profit = max(max_profit, new_profit)

        return max_profit
```



Optimized Approach





- 1. Calculate the original profit without any change!**
- 2. Calculate the first window to get initial delta!**
- 3. Slide the window from index 1 and update the delta!**
- 4. Sum original profit + delta!**
- 5. Done!**



1. Calculate the original profit without any change!
2. Calculate the first window to get initial delta!
3. Slide the window from index 1 and update the delta!
4. Sum original profit + delta!
5. Done!

```
class Solution:
    def maxProfit(self, prices: List[int], strategy: List[int], k: int) → int:
        n = len(prices)
        half_k = k // 2

        original_profit = sum(strategy[i] * prices[i] for i in range(n))

        current_delta = 0

        for i in range(half_k):
            current_delta += (0 - strategy[i]) * prices[i]

        for i in range(half_k, k):
            current_delta += (1 - strategy[i]) * prices[i]

        max_delta = max(0, current_delta)

        for start in range(1, n - k + 1):
            left_out = start - 1
            current_delta -= (0 - strategy[left_out]) * prices[left_out]

            right_in = start + k - 1
            current_delta += (1 - strategy[right_in]) * prices[right_in]

            middle = start + half_k - 1

            current_delta -= (1 - strategy[middle]) * prices[middle]

            current_delta += (0 - strategy[middle]) * prices[middle]

            max_delta = max(max_delta, current_delta)

        return original_profit + max_delta
```



Time and Space Complexity Analysis





Space - $O(n)$

Time - $O(n^2)$

```
class Solution:
    def maxProfit(self, prices: List[int], strategy: List[int], k: int) → int:
        n = len(prices)

        def calculate_profit(strat):
            total = 0
            for i in range(len(strat)):
                total += strat[i] * prices[i]
            return total

        original_profit = calculate_profit(strategy)
        max_profit = original_profit

        for start in range(n - k + 1):
            modified_strategy = strategy.copy()

            half_k = k // 2

            for i in range(start, start + half_k):
                modified_strategy[i] = 0

            for i in range(start + half_k, start + k):
                modified_strategy[i] = 1

            new_profit = calculate_profit(modified_strategy)

            max_profit = max(max_profit, new_profit)

        return max_profit
```



Space - $O(1)$

Time - $O(n)$

```
class Solution:
    def maxProfit(self, prices: List[int], strategy: List[int], k: int) → int:
        n = len(prices)
        half_k = k // 2

        original_profit = sum(strategy[i] * prices[i] for i in range(n))

        current_delta = 0

        for i in range(half_k):
            current_delta += (0 - strategy[i]) * prices[i]

        for i in range(half_k, k):
            current_delta += (1 - strategy[i]) * prices[i]

        max_delta = max(0, current_delta)

        for start in range(1, n - k + 1):
            left_out = start - 1
            current_delta -= (0 - strategy[left_out]) * prices[left_out]

            right_in = start + k - 1
            current_delta += (1 - strategy[right_in]) * prices[right_in]

            middle = start + half_k - 1

            current_delta -= (1 - strategy[middle]) * prices[middle]

            current_delta += (0 - strategy[middle]) * prices[middle]

            max_delta = max(max_delta, current_delta)

        return original_profit + max_delta
```



Key Takeaway



1. Calculate "Delta" Instead of Total Profit
2. Baseline + Adjustment Pattern